CSCE 3600: Systems Programming

Minor Assignment 1 – sed and gawk

PROGRAM DESCRIPTION:

In this assignment, you will write sed and gawk commands to accomplish certain requested functionality. Given the many powerful features of sed and gawk, you are provided with a link to a tutorial for sed as well as gawk to assist you in completing this assignment.

Stream Editor

For help using sed, you may find the tutorial http://www.grymoire.com/Unix/Sed.html or the actual sed manual https://www.gnu.org/software/sed/manual/sed.html useful.

1. The // character sequence is often known as C++ style or single-line comments, while the /* … */ character sequence is often known as C-style or multi-line comments. As an example, assume the following **myprog.c** file that has a mix of both C-style and C++ style comments:

```
// This is a test of how this works #include
<stdio.h>

int main()

{

        // declare some variables here

        int num1 = 4;

        float num2 = 3.5;


        // print the result

        printf("The result is %f\n", num1 *
        num2); // this does it


        /* does it work? */ return 0;

}
```

Answer:

```
sed 's/\/\/\(.*\)$/\/\* \1 \*\/* g' myprog.c
```

```
Mercu@trinity MINGW64 ~/OneDrive/Desktop/CSCE 4110 - Algorithms/Minor1-SED-GAWK (main)
$  sed 's/\/\/\(.*\)$/\/\* \1 \*\//g' myprog.c
/*  This is a test of how this works  */
#include <stdio.h>
int main()
{
/*  declare some variables here  */
int num1 = 4;
float num2 = 3.5;

/*  print the result */
printf("The result is %f\n", num1 * num2); /*  this does it */

/* does it work? */ return 0;
}


/*
Write a one-line sed command that transforms all of the C++ style (i.e., single-line
comments) to the C-style (i.e., multi-line comments) so that after running the
appropriate sedcommand, the following would be output to the terminal:
*/

/*
solution: sed 's/\/\/\(.*\)$/\/\* \1 \*\/* g' myprog.c  */
*/
```

2. Consider the following file called **numbers.txt** containing some lines with numbers in scientific notation and some are not:

3.14

42

1.618

1.23e-4

6.022e23

0.007

15.45

99

20.24

1.60934

Write a one-line sed command (You can have multiple sed commands piped together) that prints out only the lines containing numbers and decimal numbers greater than 15 so that after running the appropriate sed command, the following would be output to the terminal:

42
15.45
99
20.24

You may write the sed command-line in the space provided below and ensure that this Word document (with your solutions) is submitted to Canvas.

Answer: sed -n '/^[0-9]\+\(\.[0-9]\+\)\?$/p' 'numbers (1).txt' |
sed -n '/^\(15\.[1-9][0-9]*\|1[6-9]\(\.[0-9][0-9]*\)\?\|[2-9][0-9]\(\.[0-9][0-9]*\)\?\)$/p'

```
Mercu@trinity MINGW64 ~/OneDrive/Desktop/CSCE 4110 - Algorithms/Minor1-SED-GAWK (main)
$ sed -n '/^[0-9]\+\(\.[0-9]\+\)\?$/p' 'numbers (1).txt' |
sed -n '/^\(15\.[1-9][0-9]*\|1[6-9]\(\.[0-9][0-9]*\)\?\|[2-9][0-9]\(\.[0-9][0-9]*\)\?\)$/p'
42
15.45
99
20.24
```

3. Consider the following file called **phone.txt** containing some arbitrary airline phone numbers as follows:

(866) 879-7647
(888) 474-7424
(371) 670-6006
(866) 266-5588
(844) 415-3955
(800) 237-2747
(800) 667-2747

Write a complete sed script called **minor1.sed** that will encrypt the phone data according to the following:

1. *Preprocessing*:
    a. Remove the parentheses from each phone number.
    b. Remove any whitespace from each phone number.
    c. Remove any dashes (i.e., '-') from each phone number.
2. *Transposition* (i.e., rearrange the order of individual characters):
    a. Swap the third and eighth number in each phone number.
3. *Substitution* (i.e., replace characters by other characters):
    a. Substitute each number in the phone number with random letters, being sure to use an even mix of upper- and lower-case letters.
4. *Append extra data*:
    a. Append the first 10 letters of your first and last name AFTER the third phone number in the file. If your first and last names combined are shorter than 10 letters, use letters from your middle name as well. In this file, for example, my sed script should print the following:

$ **sed -f minor1.sed phone.txt**
FSSFDGDStD
FFttDtDFet
rDqSDqSwqS
MarkThomps

FSAeSSASFF
FtGtwArtAA
FqDerDeqtD
FqDSSDeqtD

Answer:

```
# Remove parentheses
s/(//g
s/)//g
# Remove spaces
s/ //g
# Remove dashes
s/-//g
# Rearrange digits: reverse the 10-digit phone number
s/\([0-9]\)\([0-9]\)\([0-9]\)\([0-9]\)\([0-9]\)\([0-9]\)\([0-9]\)\([0-9]\)\([0-9]\)\([0-9]\)/\10\9\8\7\6\5\4\3\2\1/
# Transliterate digits to letters
y/0123456789/aAbBcCdDeE/
# Append extra data after the third line
3a\
MarkThomps
```

```
Mercu@trinity MINGW64 ~/OneDrive/Desktop/CSCE 4110 - Algorithms/Minor1-SED-GAWK (main)
$  sed -f minor1.sed phone.txt
eacdDEDedde
eabcDcDceee
BaaadaDdADB
MarkThomps
eaeCCddbdde
eaCEBCAccce
eacDbDBbaae
eacDbDddaae
```

GAWK

For help using gawk, you may find the tutorial http://www.grymoire.com/Unix/Awk.html or the gawk manual https://www.gnu.org/software/gawk/manual/gawk.html useful. a)

1. Consider the following file called coords.txt

x1	y1	x2	y2
0.0	0.0	0.4	1.0
0.5	0.1	-0.25	0.9
0.75	0.4	-2.5	2.2
1.0	1.3	-1.0	4.2

Each field in this file is separated by a tab and each record is separated by a newline character. **gawk** is a very powerful utility; it can make calculations as well as support branching statements and function calls. For this part, you will write a one-line gawk program file where the program and input file are provided as command-line arguments to compute the distance between each pair of points Hint: gawk supports the sqrt function! Obviously, you need to skip the first line (i.e., the header) and only print out four values, the first of which is 1.07703.

Answer: gawk 'NR > 1 { print sqrt(($3-$1)^2 + ($4-$2)^2) }' coords.txt
1.07703
1.09659
3.71517
3.52278

```
Mercu@trinity MINGW64 ~/OneDrive/Desktop/CSCE 4110 - Algorithms/Minor1-SED-GAWK (main)
$ gawk 'NR > 1 { print sqrt(($3-$1)^2 + ($4-$2)^2) }' coords.txt
1.07703
1.09659
3.71517
3.52278
```

2. Consider the following file called **salaries.csv**: uid,loc,div,year,sal
   dgarcia,WDC,fin,2023,102900
   ejones,SEA,sales,2017,56450
   dlarkin,RCH,rsrch,2020,78435
   ccartwright,RCH,acct,2022,48775
   mlesko,OTT,rsrch,2024,93870
   criggs,WDC,sales,2021,62334
   jpetrovski,RCH,acct,2017,68348
   tchepregi,SEA,rsrch,2024,84290
   amurchin,OTT,sales,2019,49038
   gcarlson,RCH,fin,2019,70000
   swelsh,WDC,sales,2024,39876

Each field in this file is separated by a comma and each record is separated by a newline character. For this file, you will write a one-line gawk command-line to filter records for employees who earn $50,000 or more in 2024. Specifically, you will print out only the userid (i.e., uid), location (i.e., loc), and salary (i.e., sal) for those employees working in 2024 who earned a salary of $50,000 or more. You may write the gawk command-line in the space provided below and ensure that this Word document (with your solutions) is submitted to Canvas.

Answer:

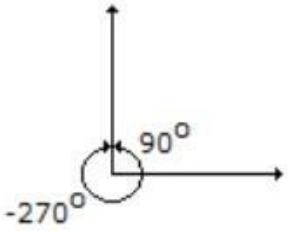
gawk -F',' '$4 == 2024 && $5 >= 50000 { print $1, $2, $5 }' salaries.csv

```
Mercu@trinity MINGW64 ~/OneDrive/Desktop/CSCE 4110 - Algorithms/Minor1-
SED-GAWK (main)
$ gawk -F',' '$4 == 2024 && $5 >= 50000 { print $1, $2, $5 }' salaries.
csv
mlesko OTT 93870
tchepregi SEA 84290
```

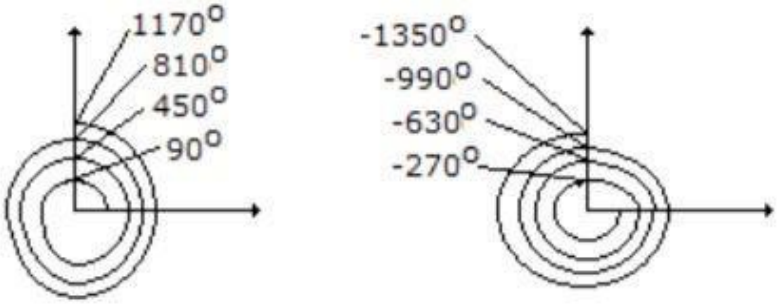
3. Consider a list of angles in degrees, such as the following **angles.txt** file:

Degrees
135
90
1860
-45
0
-270
1170
-180
-795
-630

When two angles have the same initial and terminal sides, they are called coterminal. This means that angles of 90° and -270° are coterminal as the following figure shows:



In fact, all the following angles shown in the figure below are coterminal:



We want to make sure that all angle values in degrees are between $0° \leq x < 360°$. For any file containing a list of angles in degrees with the Degrees header line at the top (as in **angles.txt**), write a complete **gawk** program that prints out the original value of the angle in the file as well as its coterminal value between $0° \leq x < 360°$. In this file, for example, the gawk program should print the following:

$ gawk -f minor1_1.gawk angles.txt

135	135
90	90
1860	60
-45	315
0	0
-270	90
1170	90
-180	180
-795	285
-630	90

Formatting properly in columns as shown is required. Write a scripts minor1_1.gawk.

```
Mercu@trinity MINGW64 ~/OneDrive/Desktop/CSCE 4110 - Algorithms/Minor1-SED-GAWK (main)
$  gawk -f minor1_1.gawk angles.txt
Degrees Coterminal
   135  135
    90  90
  1860  60
   -45  315
     0  0
  -270  90
  1170  90
  -180  180
  -795  285
  -630  90
```

4. Consider the following file called grades.txt:

```
Last        First       T1    T2    T3
----------  -----       ----  ----  ----
Adams       Will        86    77    74
Bell        Adam        98    83
Evans       Kris        92    87    96
Jackson     Ben         68    75    82
Morrow      Claire      51    62    66
Pratt       Joseph      38
Smith       Tommy       99    83    94
Welsh       Jenny       62    43    54
```

Each line (i.e., record) contains the last name, then first name, followed by one to three test scores between 0 and 99 similar to what is shown in the file above, separated by tab. A newline character separates each record. Write a single gawk program file called **minor1_2.gawk** that will read the grades.txt file and calculate the grades for each student listed, where 89.5 or greater is an “A”, 79.5 or greater is a “B”, etc. You will print out the entire record and add their grade. If a student is missing any scores, however, he/she will automatically earn an “F”. Similarly, you will print out his/her entire record, substituting “--" in place of the missing field, along with their grade and a note indicating that the student has missing scores. Obviously, you need to skip the first two lines that contain header information. In this file, for example, the gawk program should print the following:
$ gawk -f minor1_2.gawk grades.txt

```
Adams       Will        86    77    74 => C
Bell        Adam        98    83    -- => F (missing scores)
Evans       Kris        92    87    96 => A
Jackson     Ben         68    75    82 => C
Morrow      Claire      51    62    66 => D
Pratt       Joseph      38    --    -- => F (missing scores)
Smith       Tommy       99    83    94 => A
Welsh       Jenny       62    43    54 => F
```

Formatting properly in columns as shown is required. This gawk program file will be submitted to Canvas.

```
$  gawk -f minor1_2.gawk grades.txt
Adams    Will     86  77  74   C
Bell     Adam     98  83  --   F (missing scores)
Evans    Kris     92  87  96   A
Jackson  Ben      68  75  82   C
Morrow   Claire   51  62  66   D
Pratt    Joseph   38  --  --   F (missing scores)
Smith    Tommy    99  83  94   A
Welsh    Jenny    62  43  54   F
```

REQUIREMENTS:

- Your sed script and gawk program files should include **your name and EUID** at the top of the file. No other comments are needed in these files.
- For the gawk and sed commands, test out your results on real files on our CSE machines (e.g., cell01, cell02, …, cell06), to make sure that they indeed work. Your solution to the one-line sed script and gawk program can be typed (or copied and pasted) to this document and will be submitted to Canvas.
- Your program will be graded based largely on whether it works correctly on the CSE machines (e.g., cell01, cell02, …, cell06), so you should make sure that your program runs on a CSE machine. Please include any special instructions required to run your sed script and gawk program.
- This is an individual programming assignment that must be the sole work of the individual student. Any instance of academic dishonesty will result in a grade of “F” for the course, along with a report filed into the Academic Integrity Database.

SUBMISSION:

- You will electronically submit this file with your typed one-line solutions for sed and gawk along with your sed script minor1.sed and gawk program minor1_1.gawk, minor1_2.gawk to the Minor 1 dropbox in Canvas by the due date and time